

LAPORAN ALJABAR LINEAR
APLIKASI NILAI EIGEN DAN EIGEN FACE
PADA PENGENALAN WAJAH



DISUSUN OLEH :

PHOEBE THEODORE BEATRICE	L0125060
RESPANANDA AYUNING TYAS	L0125064
LUCYANA DORA NOVIYANTI	L0125121

PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI DAN SAINS DATA
UNIVERSITAS SEBELAS MARET

2026

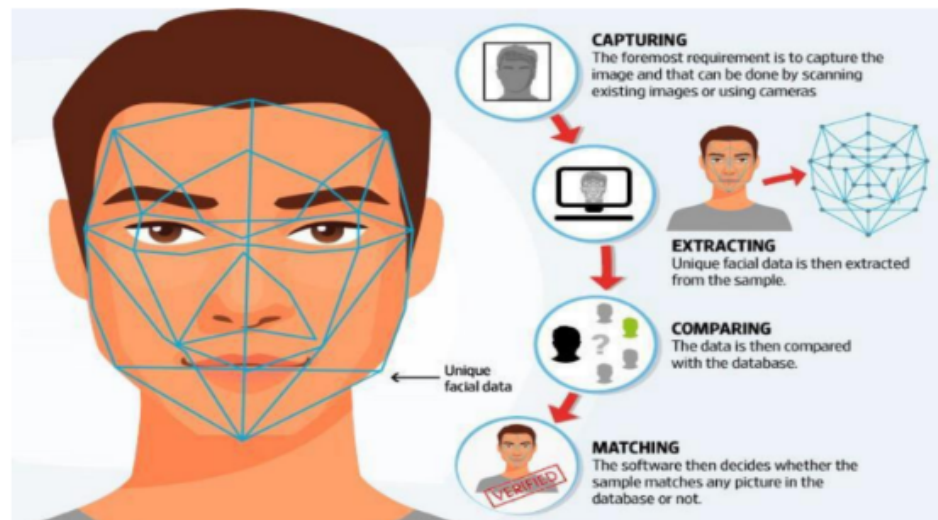
DAFTAR ISI

BAB I.....	2
A. DESKRIPSI UMUM.....	2
B. SPESIFIKASI TUGAS.....	4
BAB II.....	6
A. PERKALIAN MATRIKS.....	6
B. NILAI EIGEN.....	6
C. VEKTOR EIGEN.....	7
D. EIGENFACE.....	7
BAB III.....	9
A. LIBRARY PROGRAM.....	9
B. FILE KODE PROGRAM.....	10
C. ALGORITMA PROGRAM.....	19
BAB IV.....	21
A. FOTO ORANG YANG DITES ADA PADA DATASET DENGAN FOTO YANG SAMA.....	21
B. FOTO ORANG YANG DITES ADA PADA DATASET DENGAN FOTO YANG BERBEDA.....	21
C. FOTO ORANG YANG DITES TIDAK ADA PADA DATASET.....	22
BAB V.....	23
A. KESIMPULAN.....	23
B. SARAN.....	23
C. REFLEKSI.....	23
DAFTAR REFERENSI.....	25
LAMPIRAN.....	26

BAB I DESKRIPSI MASALAH

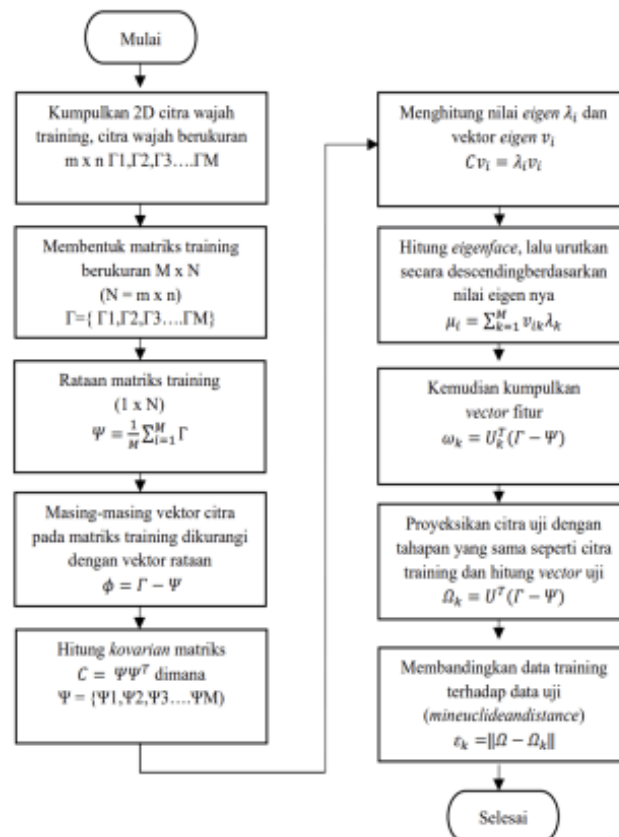
A. DESKRIPSI UMUM

Pengenalan wajah (*Face Recognition*) merupakan teknologi biometrik yang bisa dipakai untuk mengidentifikasi wajah seseorang untuk berbagai kepentingan khususnya keamanan. Program pengenalan wajah melibatkan kumpulan citra wajah yang sudah disimpan pada database lalu berdasarkan kumpulan citra wajah tersebut, program dapat mempelajari bentuk wajah lalu mencocokkan antara kumpulan citra wajah yang sudah dipelajari dengan citra yang akan diidentifikasi. Alur proses sebuah sistem pengenalan wajah diperlihatkan pada gambar di bawah ini.



Terdapat berbagai teknik untuk memeriksa citra wajah dari kumpulan citra yang sudah diketahui seperti jarak Euclidean dan *cosine similarity*, principal component analysis (PCA), serta Eigenface. Pada Tugas ini, akan dibuat sebuah program pengenalan wajah menggunakan Eigenface.

Sekumpulan citra wajah akan digunakan dengan representasi matriks. Dari representasi matriks tersebut akan dihitung sebuah matriks Eigenface. Program pengenalan wajah dapat dibagi menjadi 2 tahap berbeda yaitu tahap training dan pencocokkan. Pada tahap training, akan diberikan kumpulan data set berupa citra wajah. Citra wajah tersebut akan dinormalisasi dari RGB ke *Grayscale* (matriks), hasil normalisasi akan digunakan dalam perhitungan eigenface. Seperti namanya, matriks eigenface menggunakan eigenvector dalam pembentukannya. Berikut merupakan langkah rinci dalam pembentukan eigenface Flowchart Algoritma eigenface :



Pada tahapan akhir, akan ditemui gambar dengan euclidean distance paling kecil maka gambar tersebut yang dikenali oleh program paling menyerupai test face selama nilai kemiripan di bawah suatu nilai batas. Jika nilai minimum di atas nilai batas maka dapat dikatakan tidak terdapat citra wajah yang mirip dengan test face

Program dibuat dengan Bahasa Python dengan memanfaatkan sejumlah library di OpenCV (Computer Vision) atau library pemrosesan gambar lainnya (contoh PIL). Fungsi untuk mengekstraksi fitur dari sebuah citra wajah tidak perlu anda buat lagi, tetapi menggunakan fungsi ekstraksi yang sudah tersedia di dalam library. Fungsi Eigen dilarang import dari library dan harus diimplementasikan, sedangkan untuk operasi matriks lainnya silahkan menggunakan library.

Kode program untuk ekstraksi fitur dapat dibaca pada artikel ini: Feature extraction and similar image search with OpenCV for newbies, pada laman: <https://medium.com/machine-learning-world/feature-extraction-and-similar-image-search-with-opencv-for-newbies-3c59796bf774>

Nilai batas kemiripan citra test face dapat ditentukan oleh pembuat program melalui percobaan. Berikut merupakan referensi pengenalan wajah dengan metode eigenface: <https://jurnal.untan.ac.id/index.php/jcskommipa/article/download/9727/9500>

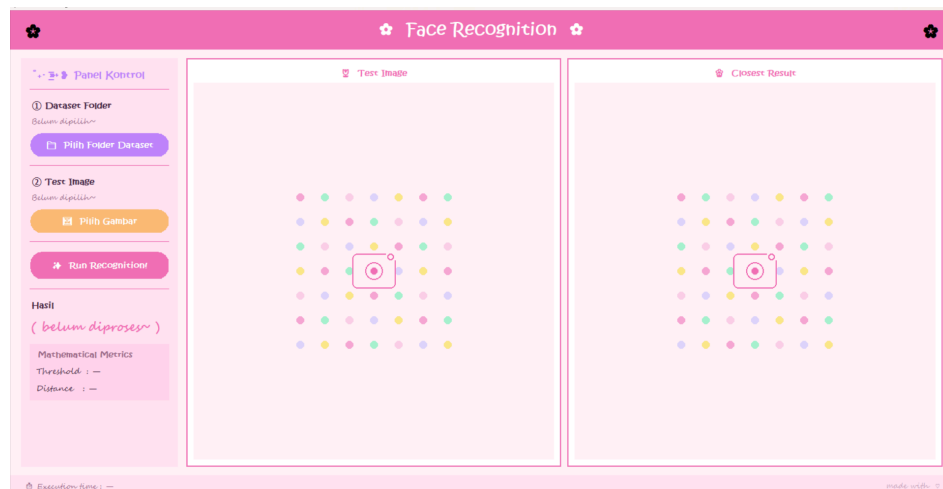
<https://www.geeksforgeeks.org/ml-face-recognition-using-eigenfaces-pca-algorithm>

PENGUNAAN PROGRAM

Berikut ini adalah input yang akan dimasukkan pengguna untuk eksekusi program

1. **Folder dataset**, berisi *folder* atau *directory* yang berisi kumpulan gambar yang digunakan sebagai *training image*.
2. **File gambar**, berisi *file* gambar input yang ingin dikenali dengan format *file* yang bebas selama merupakan format untuk gambar.

Tampilan layout dari aplikasi web yang kami bangun kurang lebih adalah sebagai berikut.



Warna ungu, orange, dan pink menunjukkan komponen yang dapat di klik. Tampilan GUI dibuat semenarik mungkin selama mencakup seluruh informasi pada layout yang diberikan di atas. Kreativitas menjadi salah satu komponen penilaian.

B. SPESIFIKASI TUGAS

Membuat program pengenalan wajah dalam Bahasa Python berbasis GUI dengan spesifikasi sebagai berikut:

1. Program menerima input *folder dataset* dan sebuah gambar citra wajah.
2. Basis data wajah dapat diunduh secara mandiri melalui <https://www.kaggle.com/datasets/hereisburak/pins-face-recognition>
3. Program menampilkan gambar citra wajah yang dipilih oleh pengguna.
4. Program melakukan pencocokan wajah dengan koleksi wajah yang ada di folder yang telah dipilih. Metrik untuk pengukuran kemiripan menggunakan eigenface + jarak *euclidean*.
5. Program menampilkan 1 hasil pencocokan pada dataset yang paling dekat dengan gambar input atau memberikan pesan jika tidak didapatkan hasil yang sesuai.

6. Program menghitung jarak *euclidean* dan nilai eigen & vektor eigen yang ditulis sendiri. Tidak boleh menggunakan fungsi yang sudah tersedia di dalam *library* atau Bahasa Python.

BAB II TEORI SINGKAT

A. PERKALIAN MATRIKS

Perkalian matriks melakukan perkalian pada setiap elemen di setiap baris matriks pertama dengan elemen elemen yang bersesuaian pada setiap kolom matriks kedua dan menjumlahkannya, perkalian jenis ini disebut perkalian titik dimana hal serupa didapatkan pada perkalian titik untuk vektor. Jumlah baris dan kolom menentukan elemen mana dari matriks produk yang dihasilkan. Suatu matriks A berukuran $m \times n$ dapat dikalikan dengan matriks B berukuran $n \times p$. Hasil perkalian akan menghasilkan matriks baru berukuran $m \times p$.

Setiap elemen pada hasil matriks diperoleh dari penjumlahan hasil kali elemen-elemen yang bersesuaian antara baris dan kolom. Perkalian matriks banyak digunakan dalam transformasi data, grafika komputer, kecerdasan buatan, dan pengolahan citra digital.

Contoh:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \quad B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

Maka:

$$A \times B = \begin{bmatrix} (1 \times 5) + (2 \times 7) & (1 \times 6) + (2 \times 8) \\ (3 \times 5) + (4 \times 7) & (3 \times 6) + (4 \times 8) \end{bmatrix} = \begin{bmatrix} 19 & 22 \\ 43 & 50 \end{bmatrix}$$

B. NILAI EIGEN

Nilai eigen menyatakan nilai karakteristik dari sebuah matriks yang berukuran $n \times n$. Nilai eigen (*eigenvalue*) adalah suatu nilai skalar yang menunjukkan perubahan skala suatu vektor setelah dikenai transformasi matriks. Kata 'Eigen' berasal dari bahasa Jerman yang berarti 'karakteristik'. Oleh karena itu, ini adalah nilai karakteristik yang menunjukkan faktor yang merentangkan vektor eigen ke arahnya. Ini tidak melibatkan perubahan arah vektor kecuali jika nilai eigennya negatif. Ketika nilai eigennya negatif, arahnya justru sebaliknya. Nilai eigen biasanya dilambangkan dengan λ . Jika A adalah matriks persegi dan x adalah vektor non-nol, maka nilai eigen A dari matriks A didefinisikan melalui persamaan:

$$A \cdot x = \lambda \cdot x$$

Persamaan ini menunjukkan bahwa ketika matriks A diterapkan pada vektor x , hasilnya adalah vektor yang searah dengan x tetapi diperbesar atau diperkecil berdasarkan faktor skala A . Untuk menemukan nilai eigen, kita harus menyelesaikan persamaan karakteristik:

$$\det(A - \lambda I) = 0$$

Di mana I adalah matriks identitas. Determinan ini menghasilkan polinomial yang memiliki derajat sama dengan dimensi matriks A , dan solusi dari polinomial tersebut memberikan nilai-nilai eigen. Nilai eigen banyak digunakan dalam analisis data, sistem dinamis, machine learning, dan pengolahan citra.

C. VEKTOR EIGEN

Vektor eigen (eigenvector) adalah vektor yang arahnya tetap setelah mengalami transformasi oleh suatu matriks. Walaupun panjang vektor dapat berubah, arah vektor tidak berubah. Sederhananya, vektor eigen adalah vektor yang terkait dengan nilai eigen.

Jika suatu matriks A dikalikan dengan vektor v dan hasilnya hanya berupa kelipatan dari vektor tersebut, maka v disebut vektor eigen. Persamaannya bisa dituliskan sebagai:

$$Av = \lambda v$$

dengan:

- v = vektor eigen,
- λ = nilai eigen.

Sebagai contoh, jika kita memiliki nilai eigen λ dari matriks A , maka vektor eigen yang terkait adalah vektor v yang memenuhi persamaan

$$(A - \lambda I) \cdot v = 0$$

Vektor eigen memberikan informasi penting tentang struktur dan perilaku sistem yang direpresentasikan oleh matriks. Secara singkat, nilai eigen adalah skalar yang menunjukkan faktor perubahan skala ketika suatu matriks diterapkan pada vektor, sementara vektor eigen adalah vektor yang tidak berubah arah di bawah transformasi matriks tersebut. Vektor eigen digunakan untuk menentukan arah dominan pada data dan menjadi dasar berbagai metode reduksi dimensi seperti PCA (*Principal Component Analysis*).

D. EIGENFACE

Eigenface adalah metode pengenalan wajah yang menggunakan konsep nilai eigen dan vektor eigen. Dinamakan eigenface karena merupakan vektor eigen dari matriks kovarian yang menyimpan informasi citra wajah. Bentuk wajah direpresentasikan secara matematis sebagai kombinasi linier dari eigen heads. Metode eigenface ini sangat mengesankan. Pengenalan wajah menggunakan kamera untuk menangkap gambar wajah dan membandingkannya dengan database yang sudah ada. Metode yang

dijelaskan adalah metode eigenface yang menggunakan webcam untuk menangkap gambar secara real-time. Metode ini bekerja dengan mengubah kumpulan gambar wajah menjadi sekumpulan vektor, lalu mencari ciri utama wajah menggunakan PCA (*Principal Component Analysis*).

BAB III

IMPLEMENTASI PROGRAM

A. LIBRARY PROGRAM

1. Library numpy

NumPy (*Numerical Python*) adalah library Python yang berfokus pada *scientific computing*. NumPy digunakan untuk operasi matematika dan pengolahan array/matriks. Numpy menyediakan fungsi yang siap pakai untuk memudahkan kita melakukan perhitungan saintifik seperti matriks, aljabar, statistik, dan sebagainya. Pada program Eigenface, library ini sangat penting karena proses pengenalan wajah menggunakan perhitungan matriks dan vektor eigen. Numpy memungkinkan program melakukan operasi matriks dengan lebih cepat dan efisien, seperti menghitung rata-rata wajah, mengubah gambar menjadi vektor, serta melakukan perhitungan eigenvalue dan eigenvector yang menjadi inti dari metode Eigenface.

2. Library os

Library os digunakan untuk menghubungkan program dengan sistem operasi, terutama dalam membaca dan mengelola file dataset wajah. Pada program face recognition, biasanya terdapat banyak folder dan gambar yang harus diproses secara otomatis. Library ini membantu program menelusuri folder, membaca nama file gambar, mengambil lokasi file, dan mengatur path dataset agar program dapat memuat semua gambar tanpa harus ditulis satu per satu secara manual. Dengan os, program dapat bekerja lebih fleksibel karena mampu membaca seluruh isi folder dataset secara otomatis.

3. Library cv2

Library cv2 dari OpenCV digunakan untuk pengolahan citra digital dan computer vision. Library ini berfungsi membaca gambar wajah, mengubah ukuran gambar, mengkonversi gambar menjadi grayscale, hingga melakukan deteksi wajah. Dalam metode Eigenface, gambar biasanya diubah menjadi grayscale agar proses perhitungan menjadi lebih sederhana karena hanya memproses intensitas cahaya tanpa warna. OpenCV juga membantu proses cropping wajah sehingga bagian yang diproses benar-benar fokus pada area wajah dan bukan latar belakang gambar.

4. Library pillow

Library pillow atau PIL digunakan untuk manipulasi gambar dan membantu pengelolaan tampilan citra pada aplikasi. Library ini memungkinkan program membuka gambar, mengubah ukuran gambar, mengubah format file gambar, dan menampilkan gambar pada GUI. Pada banyak program GUI berbasis Python,

pillow sering digunakan bersama Tkinter karena format gambar dari OpenCV kadang tidak langsung kompatibel dengan tampilan GUI. Jadi library ini berfungsi sebagai penghubung agar gambar dapat ditampilkan dengan baik pada aplikasi desktop.

5. Library tkinter

Library tkinter digunakan untuk membuat antarmuka grafis atau GUI pada program. Dengan library ini, program face recognition tidak hanya berjalan di terminal, tetapi memiliki tampilan aplikasi yang lebih interaktif dan mudah digunakan. Library ini memungkinkan pembuatan window aplikasi, tombol upload gambar, label teks, kotak hasil pengenalan wajah, dan area tampilan gambar. Dengan adanya GUI, pengguna dapat menggunakan program dengan lebih mudah tanpa harus mengetik perintah secara manual.

6. Library time

Library time digunakan untuk mengatur dan mengukur waktu dalam program. Pada project ini, library time biasanya dipakai untuk menghitung lama proses pengenalan wajah atau waktu eksekusi program. Misalnya program dapat mencatat berapa detik yang dibutuhkan sistem untuk mendeteksi dan mengenali wajah dari sebuah gambar. Fungsi ini berguna untuk mengevaluasi performa sistem, terutama ketika jumlah dataset semakin besar dan proses komputasi menjadi lebih berat.

B. FILE KODE PROGRAM

1. File data_loader.py

```
1  import os
2  import numpy as np
3  from helper import detect_and_crop_face
4
5  def load_dataset(dataset_path):
6      images = []
7      labels = []
8      original_paths = []
9
10     # Agar program mampu menelusuri seluruh folder dan subfolder
11     for root, dirs, files in os.walk(dataset_path):
12         for file in files:
13             if file.lower().endswith(('.png', '.jpg', '.jpeg')):
14                 path = os.path.join(root, file)
15
16                 #Agar nama folder atau file dapat digunakan sebagai label pada program
17                 label = os.path.basename(root) if root != dataset_path else os.path.splitext(file)[0]
18
19                 face_img = detect_and_crop_face(path)
20                 if face_img is not None:
21                     #Mengubah gambar 2D menjadi vektor 1D
22                     flattened = face_img.flatten()
23                     images.append(flattened)
24                     labels.append(label)
25                     original_paths.append(path)
26
27     # Untuk mengonversi ke numpy array
28     # X merupakan matriks dengan baris sebagai sampel gambar
29     X = np.array(images, dtype=np.float32)
30     return X, labels, original_paths
```

Kode tersebut digunakan untuk memuat dataset gambar wajah dari sebuah folder, lalu menyiapkannya agar bisa diproses dalam sistem pengenalan wajah menggunakan metode seperti Eigenface atau PCA. Program bekerja dengan membaca semua gambar yang ada di dalam folder dataset beserta subfoldernya, mendeteksi wajah pada setiap gambar, memotong bagian wajah, lalu mengubah gambar wajah tersebut menjadi data numerik berbentuk vektor sehingga dapat dihitung secara matematis oleh komputer.

Bagian awal, program mengimpor library `os`, `numpy`, dan fungsi `detect_and_crop_face` dari file. Library `os` dipakai untuk mengakses folder dan file di komputer, sehingga program dapat menelusuri isi dataset secara otomatis. Fungsi `detect_and_crop_face` berfungsi untuk mendeteksi area wajah pada gambar lalu memotong bagian wajah saja agar proses pengenalan lebih fokus dan akurat.

Fungsi `load_dataset(dataset_path)` dibuat untuk membaca seluruh dataset gambar dari folder yang diberikan melalui parameter `dataset_path`. Bagian `os.walk(dataset_path)` digunakan agar program dapat menelusuri semua folder dan subfolder di dalam dataset secara otomatis. Kemudian program memeriksa setiap file menggunakan `file.lower().endswith(...)` untuk memastikan hanya file gambar dengan format PNG, JPG, atau JPEG yang diproses. Jika file sesuai, program membuat path lengkap gambar menggunakan `os.path.join(root, file)` agar gambar dapat diakses.

Setelah itu fungsi `detect_and_crop_face(path)` dijalankan untuk mendeteksi wajah pada gambar. Jika wajah berhasil ditemukan, gambar wajah akan dipotong sehingga hanya area wajah yang digunakan. Hal ini untuk membantu meningkatkan akurasi karena bagian yang tidak penting seperti latar belakang tidak ikut diproses.

`face_img.flatten()` digunakan untuk mengubah gambar dua dimensi menjadi satu dimensi. Awalnya gambar tersusun dalam bentuk matriks piksel dengan baris dan kolom, tetapi metode Eigenface membutuhkan data dalam bentuk vektor. Karena itu semua piksel diratakan menjadi satu baris panjang agar dapat dihitung menggunakan operasi matriks.

Data wajah yang sudah diubah menjadi vektor kemudian dimasukkan ke list `images`, label dimasukkan ke `labels`, dan path asli gambar dimasukkan ke `original_paths`.

Pada bagian akhir, `np.array(images, dtype=np.float32)` digunakan untuk mengubah list gambar menjadi array NumPy bertipe

float32. Hasil akhirnya disimpan dalam variabel X, yaitu matriks data wajah di mana setiap baris merepresentasikan satu gambar wajah. Fungsi kemudian mengembalikan X, labels, dan original_paths agar dapat digunakan pada proses selanjutnya seperti training Eigenface atau pengenalan wajah.

2. File eigen_math.py

```

1  import numpy as np
2
3  def hitung_eigen(L, k):
4      num_samples = L.shape[0]
5      A = L.copy().astype(float)
6      eigenvalues = []
7      eigenvectors = []
8
9      for _ in range(k):
10         v = np.ones(num_samples)
11         v = v / np.linalg.norm(v) # Normalisasi vektor
12
13         # Power iteration loop untuk mencari arah eigenvector
14         for _ in range(50):
15             # Mengganti looping for-for-for dengan perkalian matriks '@'
16             Av = A @ v
17             norm_Av = np.linalg.norm(Av)
18
19             if norm_Av < 1e-9:
20                 break
21             v = Av / norm_Av
22
23         # Hitung Eigenvalue (Rayleigh Quotient)
24         lam = v.T @ A @ v
25
26         eigenvalues.append(lam)
27         eigenvectors.append(v)
28
29         # Deflasi Matriks dengan menggunakan operasi outer product yang cepat
30         A -= lam * np.outer(v, v)
31
32     return np.array(eigenvalues), np.array(eigenvectors).T
33
34
35 def calculate_eigenfaces(X, num_components=None):
36     num_samples, num_features = X.shape
37
38     if num_components is None:
39         num_components = num_samples - 1
40
41     # Menghitung rata-rata wajah
42     mean_face = np.mean(X, axis=0)
43
44     # Mengurangi setiap wajah dengan rata-rata
45     phi = X - mean_face
46
47     # Menghitung matriks kovarians (L = phi @ phi.T) ukuran N x N
48     L = np.dot(phi, phi.T)
49
50     # Memetakan eigen value dan eigen vector
51     eigenvalue, eigenvector_L = hitung_eigen(L, num_components)
52
53     # Konversi eigen vector dari matriks L agar kembali ke ruang fitur asli
54     eigenvector = np.dot(phi.T, eigenvector_L)
55
56     # Normalisasi panjang eigenvectors menjadi 1
57     norm = np.linalg.norm(eigenvector, axis=0)
58     norm[norm == 0] = 1
59     eigenvector = eigenvector / norm
60
61     # Ambil top 'k' komponen
62     top_eigenvector = eigenvector[:, :num_components]
63
64     return mean_face, top_eigenvector

```

Kode tersebut digunakan untuk menghitung eigenface dari kumpulan data wajah menggunakan metode PCA (Principal Component Analysis). Tujuan utama program ini adalah mencari ciri paling penting dari sekumpulan gambar wajah sehingga wajah dapat direpresentasikan dalam bentuk yang lebih sederhana tetapi tetap mempertahankan informasi utama. Hasil dari proses ini dipakai untuk mengenali atau membedakan wajah seseorang.

Fungsi `calculate_eigenfaces(X, num_components=None)`, parameter `X` berisi kumpulan data wajah yang sudah diubah menjadi vektor satu dimensi. Setiap baris pada `X` merepresentasikan satu gambar wajah, sedangkan setiap kolom merepresentasikan nilai piksel. Misalnya jika gambar berukuran 100×100 piksel, maka satu wajah akan menjadi vektor dengan panjang 10.000.

Bagian `if num_components is None:` digunakan untuk menentukan jumlah komponen eigenface yang akan dipakai. Jika pengguna tidak menentukan jumlahnya, program otomatis menggunakan `jumlah sampel - 1`. Karena jumlah maksimum eigenvector bermakna pada PCA tidak bisa melebihi jumlah sampel dikurangi satu. Kemudian program menghitung rata-rata seluruh wajah menggunakan `np.mean(X, axis=0)`. Hasilnya `mean_face` atau rata-rata wajah. Untuk `phi = X - mean_face` digunakan untuk mengurangi setiap data wajah dengan rata-rata wajah. Proses ini disebut centering data. Tujuannya agar data lebih fokus pada variasi antar wajah, misalnya bentuk mata, hidung, atau mulut, bukan pada nilai piksel absolut.

`L = np.dot(phi, phi.T)`, secara teori PCA membutuhkan matriks kovarians berukuran jumlah piksel $\times$ jumlah piksel. Namun, ukuran ini bisa sangat besar dan boros memori. Misal gambar 100×100 memiliki 10.000 piksel sehingga matriks kovarians menjadi 10.000×10.000 . Karena itu digunakan trik Eigenface dengan menghitung matriks yang lebih kecil yaitu `phi  $\times$  phi.T` berukuran jumlah sampel $\times$ jumlah sampel, sehingga membuat proses jauh lebih cepat dan hemat RAM.

3. File `face_reconizer.py`

```

1 import numpy as np
2 from eigen_math import calculate_eigenfaces
3 from data_loader import load_dataset
4 from helper import detect_and_crop_face
5
6 class FaceRecognizer:
7     # Untuk threshold bawaan/default
8     def __init__(self, threshold=3500.0):
9         self.threshold = threshold
10        self.mean_face = None
11        self.eigenvectors = None
12        self.projected_weights = None
13        self.labels = []
14        self.original_paths = []
15
16    def train(self, dataset_path):
17        X, self.labels, self.original_paths = load_dataset(dataset_path)
18        if len(X) == 0:
19            return False
20
21        self.mean_face, self.eigenvectors = calculate_eigenfaces(X)
22
23        # Untuk royeksikan dataset ke ruang eigen dengan bobot/weights
24        phi = X - self.mean_face
25        self.projected_weights = np.dot(phi, self.eigenvectors)
26        return True
27
28    def recognize(self, image_path):
29        if self.mean_face is None:
30            return None, 0, None, "Model belum dilatih"
31
32        face_img = detect_and_crop_face(image_path)
33        if face_img is None:
34            return None, 0, None, "Wajah tidak terdeteksi"
35
36        # Untuk vektorisasi wajah yang dites dan mengurangi mean
37        test_flattened = face_img.flatten()
38        phi_test = test_flattened - self.mean_face
39
40        # Untuk memproyeksikan wajah yang dites
41        test_weight = np.dot(phi_test, self.eigenvectors)
42
43        # Untuk menghitung euclidean distance ke semua bobot pada dataset
44        distances = np.linalg.norm(self.projected_weights - test_weight, axis=1)
45
46        min_idx = np.argmin(distances)
47        min_dist = distances[min_idx]
48
49        if min_dist < self.threshold:
50            return self.labels[min_idx], min_dist, self.original_paths[min_idx], "Dikenali"
51        else:
52            return "Unknown", min_dist, None, "Melewati Threshold"

```

File ini merupakan inti dari sistem face recognition yang menggunakan metode Eigenface. Class `FaceRecognizer` untuk melatih model menggunakan dataset wajah dan mengenali wajah baru berdasarkan kemiripan fitur wajah.

Pada bagian `__init__`, program membuat beberapa variabel penting seperti `threshold`, `mean_face`, `eigenvectors`, dan `projected_weights`. *Threshold* digunakan sebagai batas pengenalan wajah. Jika jarak kemiripan terlalu besar, wajah dianggap tidak dikenali. Variabel lain digunakan untuk menyimpan hasil perhitungan Eigenface, label nama, dan lokasi gambar asli dataset.

Fungsi `train()` digunakan untuk melatih model wajah. Program pertama-tama memuat dataset menggunakan `load_dataset()`. Jika dataset kosong, fungsi langsung mengembalikan `False`. Setelah data berhasil dimuat, program menghitung rata-rata wajah dan eigenface menggunakan `calculate_eigenfaces(X)`. Kemudian data wajah dikurangi dengan rata-rata wajah menggunakan `phi = X - self.mean_face` Tujuannya agar sistem fokus pada perbedaan ciri antar wajah. Setelah itu dataset diproyeksikan ke ruang eigen menggunakan: `self.projected_weights = np.dot(phi, self.eigenvectors)`. Hasil proyeksi ini disebut bobot atau weights, yaitu representasi numerik wajah

pada ruang Eigenface. Bobot inilah yang nantinya dibandingkan saat proses pengenalan.

Fungsi `recognize()` digunakan untuk mengenali wajah baru. Pertama program memeriksa apakah model sudah dilatih. Jika belum, program mengembalikan pesan "Model belum dilatih". Selanjutnya gambar uji diproses menggunakan `detect_and_crop_face()` untuk mendeteksi dan memotong wajah. Jika wajah tidak ditemukan, program menampilkan pesan "Wajah tidak terdeteksi". Wajah yang diuji kemudian diubah menjadi vektor satu dimensi menggunakan `flatten()`, lalu dikurangi dengan rata-rata wajah. Setelah itu wajah diproyeksikan ke ruang eigen `test_weight = np.dot(phi_test, self.eigenvectors)`. Hasil proyeksi ini dibandingkan dengan semua bobot wajah pada dataset menggunakan Euclidean Distance: `distances = np.linalg.norm(self.projected_weights - test_weight, axis=1)`. Semakin kecil jaraknya, semakin mirip wajah tersebut dengan data training. Program kemudian mencari jarak terkecil menggunakan `np.argmin()` dan `np.min()`. Jika jarak terkecil lebih kecil dari `threshold`, wajah dianggap dikenali dan program mengembalikan label nama beserta gambar yang cocok. Namun jika jaraknya melebihi `threshold`, program mengembalikan "Unknown" karena wajah dianggap tidak cocok dengan dataset yang ada.

4. File gui.py

```

1  import os
2  import tkinter as tk
3  from tkinter import filedialog, messagebox
4  import numpy as np
5  from PIL import Image, ImageTk, ImageDraw, ImageFilter
6
7  # Palet Warna
8  BG_MAIN      = "#FFF0F7"
9  BG_CARD      = "#FFFFFF"
10 BG_SIDE_BAR  = "#FFE4F3"
11 ACCENT_PINK   = "#F472B6"
12 ACCENT_LILAC = "#C084FC"
13 ACCENT_MINT   = "#588272"
14 ACCENT_PEACH  = "#FDBA74"
15 TEXT_DARK     = "#4B2C47"
16 TEXT_MID     = "#9D6B8A"
17 TEXT_LIGHT    = "#C9A5BD"
18 BTN_HOVER     = "#FCE7F3"
19
20 # Font
21 FONT_TITLE    = ("Kristen ITC", 20, "bold")
22 FONT_LABEL    = ("Kristen ITC", 10, "bold")
23 FONT_SMALL    = ("Segoe Script", 9)
24 FONT_RESULT   = ("Segoe Script", 15, "bold")
25 FONT_METRIC   = ("Segoe Script", 10)
26
27 # Helper image placeholder
28 def make_placeholder(w=280, h=280):
29     img = Image.new("RGB", (w, h), "#FFF0F7")
30     draw = ImageDraw.Draw(img)
31     import random
32     random.seed(42)
33     dots = [(c, r) for c in range(20, w, 40) for r in range(20, h, 40)]
34     colors = [{"#F9A8D4", "#DDD6FE", "#A7F3D0", "#FDE68A", "#FBCFE8"}]
35     for i, (cx, cy) in enumerate(dots):
36         col = colors[i % len(colors)]
37         draw.ellipse([cx-6, cy-6, cx+6, cy+6], fill=col, outline=None)
38     cx, cy = w//2, h//2
39     draw.rounded_rectangle([cx-35, cy-28, cx+35, cy+28], radius=8, outline=ACCENT_PINK, width=2)
40     draw.ellipse([cx-14, cy-14, cx+14, cy+14], outline=ACCENT_PINK, width=2)
41     draw.ellipse([cx-5, cy-5, cx+5, cy+5], fill=ACCENT_PINK)
42     draw.rectangle([cx+20, cy-28, cx+35, cy-18], fill="#FFF0F7")
43     draw.ellipse([cx+22, cy-28, cx+32, cy-18], outline=ACCENT_PINK, width=2)
44     return img

```


File ini untuk membuat tampilan (GUI) aplikasi face recognition menggunakan library `tkinter`. Program ini berfungsi sebagai penghubung antara pengguna dengan sistem pengenalan wajah sehingga pengguna dapat memilih dataset, memasukkan gambar uji, menjalankan proses recognition, dan melihat hasilnya secara visual.

Pada bagian awal terdapat pengaturan warna dan font untuk mempercantik tampilan aplikasi dengan tema pastel. Library `PIL` digunakan untuk mengolah dan menampilkan gambar pada GUI, sedangkan `numpy` dipakai untuk membantu pengolahan data numerik.

Fungsi `make_placeholder()` digunakan untuk membuat gambar placeholder ketika belum ada gambar yang dipilih. Gambar ini hanya sebagai tampilan awal agar area gambar tidak kosong. Class `PastelButton` dibuat untuk membuat tombol custom dengan desain bulat dan efek hover. Saat kursor diarahkan ke tombol, warna tombol akan berubah menjadi lebih terang sehingga tampilan GUI menjadi lebih interaktif.

Class utama `FaceRecognitionGUI` bertugas membangun seluruh tampilan aplikasi. Pada `__init__`, program membuat jendela utama dengan judul “Face Recognition”, mengatur ukuran layar, warna latar, dan menyimpan variabel penting seperti path dataset dan gambar uji.

Fungsi `create_widgets()` membangun bagian-bagian GUI seperti header, sidebar, panel gambar, dan footer. Sidebar berisi tombol untuk memilih folder dataset, memilih gambar uji, serta menjalankan proses recognition. Di bagian hasil, program menampilkan nama hasil pengenalan, nilai *threshold*, dan distance.

Fungsi `browse_dataset_folder()` digunakan untuk memilih folder dataset. Program akan menghitung jumlah orang dan jumlah foto yang ada di dalam dataset, lalu menampilkannya pada GUI. Jika folder tidak berisi gambar, program akan menampilkan pesan peringatan. Fungsi `browse_test_image()` digunakan untuk memilih gambar yang akan diuji. Setelah dipilih, gambar langsung ditampilkan pada panel “Test Image”.

Bagian `update_display_metrics()` digunakan untuk menampilkan hasil pengenalan wajah. Program menampilkan execution time, *threshold*, distance, dan status apakah wajah dikenali atau tidak. Jika wajah dikenali, gambar hasil yang paling mirip akan ditampilkan pada panel “Closest Result”. Jika tidak dikenali, panel hasil akan kembali menampilkan placeholder.

5. File helper.py

```
1  import cv2
2  import numpy as np
3
4  # Menggunakan model Haar Cascade bawaan OpenCV untuk deteksi wajah
5  face_cascade = cv2.CascadeClassifier(cv2.data.harcascades + 'haarcascade_frontalface_default.xml')
6
7  def detect_and_crop_face(image_path, target_size=(100, 100)):
8      # Untuk membaca gambar, mendeteksi wajah, memotong, dan meresize
9      img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
10     if img is None:
11         return None
12
13     faces = face_cascade.detectMultiScale(img, scaleFactor=1.1, minNeighbors=5, minSize=(30, 30))
14
15     if len(faces) == 0:
16         #Agar jika tidak terdeteksi, diatur ulang ke tengah
17         resized = cv2.resize(img, target_size)
18         return resized
19
20     # Untuk mengambil wajah pertama yang terdeteksi
21     (x, y, w, h) = faces[0]
22     face_crop = img[y:y+h, x:x+w]
23
24     #Untuk mengatur ulang ke ukuran standar agar vektor fitur panjangnya sama
25     face_resized = cv2.resize(face_crop, target_size)
26     return face_resized
```

File tersebut digunakan untuk mendeteksi wajah pada gambar lalu memotong bagian wajah agar dapat diproses pada sistem pengenalan wajah. Program memakai library cv2 dari OpenCV untuk pengolahan citra dan model Haar Cascade bawaan OpenCV untuk mendeteksi posisi wajah pada gambar.

Fungsi `detect_and_crop_face()` menerima input berupa path gambar dan ukuran target. Gambar dibaca dalam mode grayscale menggunakan `cv2.imread(..., cv2.IMREAD_GRAYSCALE)` agar proses lebih ringan karena hanya memakai intensitas warna abu-abu. Jika gambar gagal dibaca, fungsi akan mengembalikan `None`.

Bagian `face_cascade.detectMultiScale()` digunakan untuk mencari wajah pada gambar. Jika tidak ada wajah yang terdeteksi, gambar tetap di-resize ke ukuran standar agar masih bisa diproses. Namun jika wajah ditemukan, program mengambil wajah pertama yang terdeteksi menggunakan koordinat (x, y, w, h) lalu memotong area wajah tersebut.

Wajah hasil potongan diubah ukurannya menjadi 100×100 piksel menggunakan `cv2.resize()`. Penyeragaman ukuran ini agar semua gambar memiliki jumlah piksel yang sama sehingga dapat diubah menjadi vektor fitur dengan panjang yang konsisten pada proses Eigenface atau PCA.

6. File main.py

```
1 import tkinter as tk
2 from gui import FaceRecognitionGUI
3 from face_recognizer import FaceRecognizer
4 import time
5
6 def main():
7     root = tk.Tk()
8     recognizer = FaceRecognizer(threshold=3500.0)#Untuk batas pengenalan
9
10    def run_recognition():
11        if not app.selected_dataset_path:
12            return
13        if not app.test_image_path:
14            return
15
16        #Untuk pengatur waktu
17        start = time.time()
18        recognizer.train(app.selected_dataset_path)
19        label, dist, matched_path, status = recognizer.recognize(app.test_image_path)
20        elapsed = time.time() - start
21
22        result_label = label if label != "Unknown" else "Unknown (Tidak Dikenali)"
23        app.update_display_metrics(result_label, dist, recognizer.threshold, elapsed, matched_path)
24
25    app = FaceRecognitionGUI(root, run_callback=run_recognition)
26    root.mainloop()
27
28 if __name__ == "__main__":
29     main()
```

File ini merupakan program utama untuk menjalankan aplikasi GUI face recognition menggunakan `tkinter`. Program ini menghubungkan tampilan antarmuka dengan proses pengenalan wajah. File ini berisi kode untuk menjalankan program yang berisi dari kumpulan pemanggilan fungsi-fungsi dari file lain.

Pada awal program, `tkinter` digunakan untuk membuat jendela aplikasi, `FaceRecognitionGUI` digunakan sebagai tampilan antarmuka, sedangkan `FaceRecognizer` berisi proses training dan pengenalan wajah. Library `time` dipakai untuk menghitung lama proses pengenalan.

Fungsi `main()` membuat jendela utama dengan `tk.Tk()` lalu membuat objek `FaceRecognizer` dengan nilai `threshold=3500.0`. *Threshold* ini digunakan sebagai batas apakah wajah dianggap cocok atau tidak. Jika jarak hasil pengenalan melebihi *threshold*, wajah akan dianggap tidak dikenali.

Di dalam fungsi `run_recognition()`, program terlebih dahulu memeriksa apakah dataset dan gambar uji sudah dipilih. Jika belum, proses dihentikan. Setelah itu program mulai menghitung waktu proses menggunakan `time.time()`, lalu melakukan training dataset melalui `recognizer.train()` dan mengenali wajah menggunakan `recognizer.recognize()`.

Hasil pengenalan berupa label nama, nilai jarak kemiripan, gambar yang cocok, dan status pengenalan. Lama proses dihitung dengan mengurangi waktu akhir dan awal. Jika wajah tidak dikenali, program menampilkan tulisan "Unknown (Tidak Dikenali)".

Terakhir, hasil pengenalan ditampilkan pada GUI melalui `app.update_display_metrics()`. Perintah `root.mainloop()` digunakan agar jendela aplikasi tetap berjalan dan dapat menerima interaksi dari pengguna.

C. ALGORITMA PROGRAM

Algoritma program face recognition ini dimulai ketika pengguna memilih folder dataset dan gambar uji melalui GUI. Folder dataset berisi kumpulan foto wajah yang akan digunakan sebagai data awalnya, sedangkan gambar uji digunakan untuk dicocokkan dengan data tersebut.

Saat tombol "Run Recognition" ditekan, file `main.py` menjalankan fungsi `run_recognition()`. Fungsi ini memanggil `recognizer.train()` untuk melatih model menggunakan dataset yang dipilih.

Pada proses training, `FaceRecognizer` mengambil semua gambar dari dataset melalui fungsi `load_dataset()` pada file `data_loader.py`. Program menelusuri seluruh folder dan subfolder menggunakan `os.walk()`. Setiap file gambar yang ditemukan akan dibaca satu per satu, lalu diteruskan ke fungsi `detect_and_crop_face()` pada file `helper.py`.

Di `helper.py`, gambar diproses menggunakan Haar Cascade dari OpenCV untuk mendeteksi posisi wajah. Jika wajah ditemukan, bagian wajah dipotong lalu di-resize menjadi ukuran tetap 100×100 piksel agar semua gambar memiliki ukuran yang sama. Jika wajah tidak terdeteksi, gambar tetap di-resize supaya masih bisa diproses.

Setelah wajah berhasil dipotong, gambar diubah menjadi vektor satu dimensi menggunakan `flatten()`. Semua vektor wajah kemudian dikumpulkan menjadi matriks data `X`, di mana setiap baris merepresentasikan satu wajah.

Matriks ini kemudian dikirim ke fungsi `calculate_eigenfaces()` pada file `eigen_math.py`. Di sini program menghitung rata-rata seluruh wajah (mean face), lalu setiap wajah dikurangi dengan rata-ratanya agar sistem fokus pada perbedaan ciri wajah. Setelah itu dihitung matriks kovarians untuk mencari eigenvalue dan eigenvector menggunakan PCA.

Eigenvector dengan informasi paling penting dipilih sebagai eigenface, yaitu pola utama yang mewakili karakteristik wajah pada dataset. Semua wajah training

kemudian diproyeksikan ke ruang eigen untuk menghasilkan bobot (weights) yang menjadi representasi numerik setiap wajah.

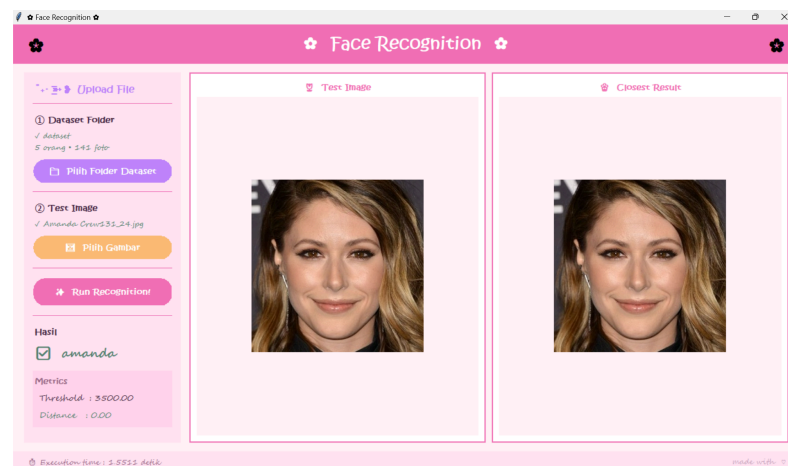
Ketika gambar uji diproses, alurnya hampir sama. Gambar dikirim lagi ke `detect_and_crop_face()` untuk dipotong dan di-resize, lalu diubah menjadi vektor dan dikurangi dengan mean face. Setelah itu gambar diproyeksikan ke ruang eigen sehingga menghasilkan bobot wajah yang uji.

Program lalu membandingkan bobot wajah uji dengan semua bobot wajah pada dataset menggunakan Euclidean Distance. Jarak terkecil menunjukkan wajah yang paling mirip. Jika jarak tersebut lebih kecil dari *threshold*, wajah dianggap dikenali dan label nama ditampilkan. Jika jarak lebih besar, hasilnya dianggap "Unknown". Lalu, hasil recognition dikirim kembali ke GUI untuk ditampilkan berupa nama hasil pengenalan, nilai distance, execution time, dan gambar wajah yang paling cocok.

BAB IV PERCOBAAN

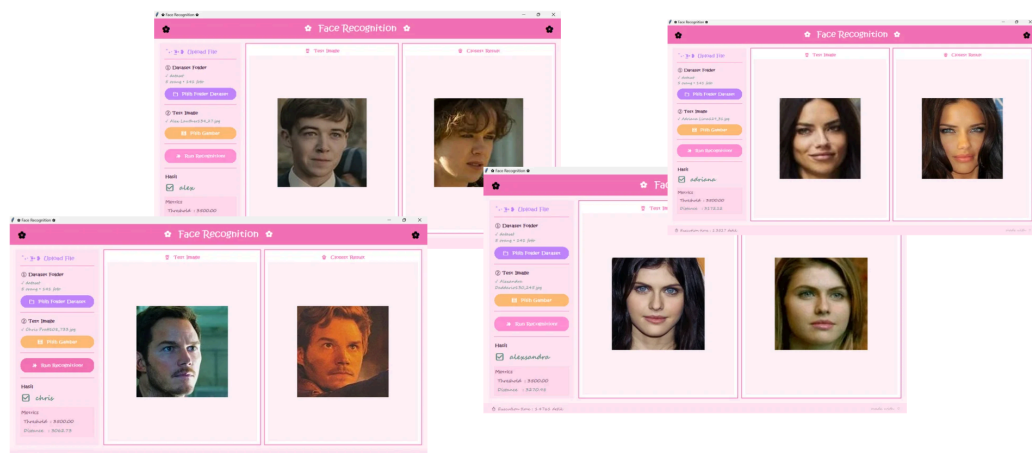
A. FOTO ORANG YANG DITES ADA PADA DATASET DENGAN FOTO YANG SAMA

Pada percobaan ini, foto yang dimasukkan sebagai *test image* juga berada pada folder dataset. Hal ini menghasilkan jarak *euclidean* terdekat berupa 0 karena foto yang ingin dites sama persis dengan apa yang ada pada dataset.



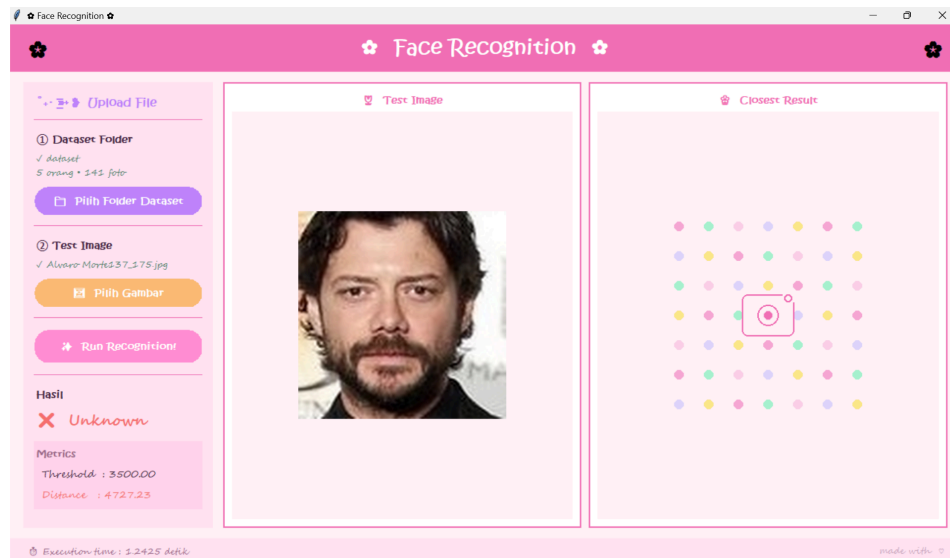
B. FOTO ORANG YANG DITES ADA PADA DATASET DENGAN FOTO YANG BERBEDA

Pada percobaan ini, foto yang dimasukkan sebagai *test image* tidak berada pada folder dataset. Namun, pada dataset hanya terdapat foto dari orang yang sama tetapi dengan gaya atau sudut pandang yang berbeda. Hal ini membuat program menampilkan foto dengan jarak *euclidean* terdekat. Jarak *euclidean* tersebut masih berada di bawah *threshold* karena berupa orang yang sama.



C. FOTO ORANG YANG DITES TIDAK ADA PADA DATASET

Pada percobaan ini, foto yang dimasukkan sebagai *test image* tidak berada pada folder dataset juga dengan foto lainnya dari orang yang sama. Hal ini menyebabkan program menemukan foto orang lain yang memiliki jarak terdekat. Namun, karena merupakan orang yang berbeda, foto dengan jarak *euclidean* terdekat melebihi *threshold* yang ditentukan sehingga tidak menampilkan foto apapun (*Unknown*).



BAB V

PENUTUP

A. KESIMPULAN

Dari hasil percobaan yang dilakukan, program pengenalan wajah menggunakan metode Eigenface berhasil dibuat dan dapat bekerja dengan baik dalam mengenali wajah berdasarkan kemiripan ciri pada dataset. Program memanfaatkan konsep aljabar linear seperti perkalian matriks, nilai eigen, dan vektor eigen untuk membentuk eigenface sebagai representasi utama wajah. Proses dimulai dari pembacaan dataset, deteksi dan pemotongan wajah, perubahan gambar menjadi vektor, hingga perhitungan PCA untuk mendapatkan fitur penting wajah.

Hasil pengujian menunjukkan bahwa sistem mampu mengenali wajah dengan akurat ketika data wajah terdapat pada dataset, baik dengan gambar yang sama maupun gambar berbeda dari orang yang sama. Selain itu, sistem juga mampu menolak wajah yang tidak terdapat pada dataset dengan memanfaatkan *threshold* dan perhitungan Euclidean Distance. Dengan adanya GUI, program menjadi lebih mudah digunakan dan lebih interaktif bagi pengguna.

B. SARAN

Program ini masih dapat dikembangkan lebih lanjut agar performa dan akurasinya meningkat. Salah satu yang dapat dilakukan adalah menambahkan metode preprocessing yang lebih baik, seperti normalisasi pencahayaan atau penghilangan noise agar sistem lebih stabil terhadap perubahan kondisi gambar.

Selain itu, dataset yang digunakan dapat diperbanyak dan dibuat lebih bervariasi dari segi pose, ekspresi, dan pencahayaan agar hasil pengenalan lebih akurat. Program juga dapat dikembangkan menjadi sistem real-time menggunakan webcam sehingga proses pengenalan wajah dapat dilakukan secara langsung.

C. REFLEKSI

Melalui proyek ini, diperoleh pemahaman yang lebih mendalam mengenai penerapan konsep aljabar linear dalam dunia nyata, khususnya pada bidang computer vision dan face recognition. Proyek ini menunjukkan bahwa konsep seperti nilai eigen, vektor eigen, dan matriks tidak hanya dipelajari secara teori, tetapi juga memiliki penerapan langsung dalam teknologi modern. Selain itu, proses pembuatan program memberikan pengalaman dalam menghubungkan berbagai komponen seperti pengolahan citra, perhitungan matematis, dan antarmuka GUI menjadi satu sistem yang utuh.

Tantangan yang dihadapi selama pengerjaan adalah mengoptimalkan performa model agar mampu mengenali wajah secara akurat, terutama dalam mengatasi variasi yang tidak terdapat dalam dataset pelatihan. Pengujian dengan data eksternal dan eksperimen dengan berbagai *threshold* mengajarkan pentingnya penyesuaian parameter dan evaluasi yang sistematis. Serta memahami proses PCA, mengatur

dataset, dan menentukan *threshold* yang sesuai, juga membantu meningkatkan kemampuan problem solving dan pemrograman secara lebih terstruktur.

DAFTAR REFERENSI

- Wibowo, F. W. (n.d.). *Handbook Aljabar Linier*. Universitas AMIKOM Yogyakarta.
http://arsipdosen.amikom.ac.id/Ferry-Wahyu-Wibowo-SSi-MCs-Dr_20240223231443_Handbook_Aljabar_Linier.pdf
- DScience Labs. (n.d.). *Nilai Eigen dan Vektor Eigen*. Bookdown.
<https://bookdown.org/dsciencelabs/aln/07-Nilai-Eigen-dan-Vektor-Eigen.html>
- Munir, R. (n.d.). *Nilai Eigen dan Vektor Eigen (Bagian 1)*. Program Studi Teknik Informatika, STEI-ITB.
<https://informatika.stei.itb.ac.id/~rinaldi.munir/AljabarGeometri/2020-2021/Algeo-18-Nilai-Eigen-dan-Vektor-Eigen-Bagian1.pdf>
- Muhardian, A. (2022). *Memahami: Apa itu Numpy pada Python?* Petani Kode.
<https://www.petanikode.com/python-numpy>

LAMPIRAN

Link Repository Github:

<https://github.com/60Phoebe/Face-reconition-dengan-eigenface>

Link Demo (YouTube):

<https://youtu.be/tt6Q8anwA14>